6.动态数组

动态数组创建对象举例

C++

```cpp
#include <iostream>
using namespace std;
class Point {
public:
Point() : x(0), y(0) {
cout<<"Default Constructor called."<<endl;
}
Point(int x, int y) : x(x), y(y) {
cout<< "Constructor called."<<endl;
}
~Point() { cout<<"Destructor called."<<endl; }
int getX() const { return x; }
int getY() const { return y; }
void move(int newX, int newY) {
x = newX;
y = newY;}
private:
int x, y;
};

int main() {
cout << "Step one: " << endl;
Point *ptr1 = new Point; //调用默认构造函数
cout<<ptr1->getX()<<endl; //输出GetX
delete ptr1; //删除对象，自动调用析构函数
cout << "Step two: " << endl;
ptr1 = new Point(1,2);
cout<<ptr1->getX()<<endl; //输出GetX
delete ptr1;
return 0;
}
```

动态数组

C++

```cpp
#include <iostream>
using namespace std;

template <typename T>
class DynamicArray {
private:
    T* array; // 指向T类型的指针
    unsigned int mallocSize; // 分配空间的大小

public:
    // 构造函数
    DynamicArray(unsigned length, const T &content) {
        mallocSize = length;
        array = new T[length];
        for (unsigned int i = 0; i < length; ++i) {
            array[i] = content;
        }
        cout << "new T[" << mallocSize << "] malloc " << mallocSize << "*" << sizeof(T) << "=" << mallocSize * sizeof(T) << " bytes memory in heap" << endl;
    }

    // 析构函数
    ~DynamicArray() {
        delete[] array;
        cout << "delete[] array free " << mallocSize << "*" << sizeof(T) << "=" << mallocSize * sizeof(T) << " bytes memory in heap" << endl;
    }
    // 返回数组容量
    unsigned int capacity() const {
        return mallocSize;
    }
    // 重载下标运算符
    T& operator[](unsigned int i) {
        return array[i];
    }
};//StudybarCommentBegin
```

```
main()
{
int length,i;
cin>> length;
DynamicArray<int> iarray(length,-1);
DynamicArray<double> darray(length,-2.1);
cout<<"capacity:"<<iarray.capacity()<<endl;
for(i=0;i<length;i++) {
cout << iarray[i] <<" ";
iarray[i] = i*1.1;
}
cout<<endl;
for(i=0;i<length;i++) {
cout << darray[i] <<" ";
darray[i] = i*1.1;
}
cout<<endl;
for(i=0;i<length;i++) {
cout << iarray[i] <<" ";
iarray[i] = i*1.1;
}
cout<<endl;
for(i=0;i<length;i++) {
cout << darray[i] <<" ";
}


}
//StudybarCommentEnd
```

动态数组—基本模板类

C++

```
#include <iostream>
using namespace std;

template <typename T>
class DynamicArray {
private:
    T* array; // 指向T类型的指针
    unsigned int mallocSize; // 分配空间的大小

public:
    // 构造函数
    DynamicArray(unsigned length, const T &content) {
        mallocSize = length;
        array = new T[length];
        for (unsigned int i = 0; i < length; ++i) {
            array[i] = content;
        }
        cout << "new T[" << mallocSize << "] malloc " << mallocSize << "*" << sizeof(T) << "=" << mallocSize * sizeof(T) << " bytes memory in heap" << endl;
    }

    // 析构函数
    ~DynamicArray() {
        delete[] array;
        cout << "delete[] array free " << mallocSize << "*" << sizeof(T) << "=" << mallocSize * sizeof(T) << " bytes memory in heap" << endl;
    }

    // 返回数组容量
    unsigned int capacity() const {
        return mallocSize;
    }

    // 重载下标运算符
    T& operator[](unsigned int i) {
        return array[i];
    }

    // 重载下标运算符 const 版本
    const T& operator[](unsigned int i) const {
        return array[i];}
};
//StudybarCommentBegin
int main()
{
int length,i;
cin>> length;

DynamicArray<int> iarray(length,-1);
DynamicArray<double> darray(length,-2.1);
const DynamicArray<char> carray(length,'c');
```

```
50
51  cout<<endl<<"capacity:"<<carray.capacity()<<endl;
52
53  for(i=0;i<length;i++) {
54  cout << carray[i] <<" ";
55  }
56  cout<<endl;
57  for(i=0;i<length;i++) {
58  cout << iarray[i] <<" ";
59  iarray[i] = i*1.1;
60  }
61  cout<<endl;
62  for(i=0;i<length;i++) {
63  cout << darray[i] <<" ";
64  darray[i] = i*1.1;
65  }
66  cout<<endl;
67  for(i=0;i<length;i++) {
68  cout << iarray[i] <<" ";
69  iarray[i] = i*1.1;
70  }
71  cout<<endl;
72  for(i=0;i<length;i++) {
73  cout << darray[i] <<" ";
74  }
75  return 0;
76  }
77  //StudybarCommentEnd
```

动态数组——基本模板 copy 构造函数

C++

```
1  #include <iostream>
2  using namespace std;
3
4  template <typename T>
5  class DynamicArray {
6  private:
7      T* array; // 指向T类型的指针
8      unsigned int mallocSize; // 分配空间的大小
9
10 public:
11     // 构造函数
12     DynamicArray(unsigned length, const T &content) {
13         mallocSize = length;
14         array = new T[length];
15         for (unsigned int i = 0; i < length; ++i) {
16             array[i] = content;
17         }
18         cout << "new T[" << mallocSize << "] malloc " << mallocSize << "*" << sizeof(T) << "=" << mallocSize * sizeof(T) << " by
   tes memory in heap" << endl;
19     }
20
21     // Copy 构造函数
22     DynamicArray(const DynamicArray<T> &anotherDA) {
23         mallocSize = anotherDA.mallocSize;
24         array = new T[mallocSize];
25         for (unsigned int i = 0; i < mallocSize; ++i) {
26             array[i] = anotherDA.array[i];
27         }
28         cout << "Copy Constructor is called";
29     }
30
31     // 析构函数
32     ~DynamicArray() {
33         delete[] array;
34         cout << endl<<"delete[] array free " << mallocSize << "*" << sizeof(T) << "=" << mallocSize * sizeof(T) << " bytes memor
   y in heap" ;
35     }
36
37     // 返回数组容量
38     unsigned int capacity() const {
39         return mallocSize;
40     }
41
42     // 重载下标运算符
43     T& operator[](unsigned int i) {
44         return array[i];
45     }
46
47     // 重载下标运算符 const 版本
48     const T& operator[](unsigned int i) const {
49         return array[i];
50     }
```

```
};
//StudybarCommentBegin
int main()
{
int length,i;
cin>> length;

DynamicArray<int> iarray(length,-1);

DynamicArray<int> iarray2(iarray);

cout<<endl;
for(i=0;i<length;i++) {
  cout << iarray[i] <<" ";
  iarray[i] = i*1.1;
}
cout<<endl;
for(i=0;i<length;i++) {
  cout << iarray2[i] <<" ";
}
return 0;
}
//StudybarCommentEnd
```